BS(CS/SE/DS/AI) FINAL PROJECT
Software Requirements Specification

Privacy-Preserving Healthcare GenAI



Product Owner

Prof Dr Saira Andleeb Gilani

Presented by:
Group ID: F-3

Student Reg#	Student Name
L1F23BSCS0192	Faisal Khalid
L1F23BSCS0199	Shah Wali
L1F23BSCS0669	Wajeeh ur Rehman
L1F23BSCS0204	M. Hammad

Faculty of Information Technology & Computer Science

University of Central Punjab

Software Requirements Specification

Version: 1.20

Product Owner:

Prof Dr Saira Andleeb Gilani

Scrum Master

Faisal Khalid

Group F-3

Team

Member Name	Roles
Faisal Khalid	KubeOps
Shah Wali	FL Security
Wajeeh ur Rehman	PipeLine/MLOps
M. Hammad	Backend Integration

Table of Contents

Table of Contents ..... **i**
Revision History & Sprint Log ..... **ii**
Previous Phases Feedback ..... **iii**
Abstract **1**
1. Introduction and Background **1**
1.1 Product Vision & Problem Context 1
1.2 Stakeholders Identification 9
2. Agile Framework Setup **10**
2.1 Epics For the Project 10
2.2 Product Backlog 11
2.3 User Stories 12
2.4 Story Points & Estimation 14
2.5 Sprint Planning & Sprint Backlog 15
3. Functional Requirements **16**
3.1 Use Case Diagram 16
4. Non-functional Requirements **18**
4.1 Reliability 18
4.2 Performance 18
4.3 Use-ability 18
4.4 Other Non-functional Requirements 18
4.4.1 Security Requirements 18
4.4.2 Scalability Requirements 19
4.4.3 Explainability & Validation Requirements 19
4.4.4 Compliance Requirements 19
5. Scrum Activities Plan **20**
5.1 Kanban Board / Task Tracking 20
5.2 Acceptance Criteria 20
5.3 Release Criteria 21
5.4 Burndown Chart & Progress Tracking 22
5.5 Backlog Refinement Plan 23
6. Risk Analysis **24**
7. References **25**
Appendix A: Glossary **26**
Appendix B: IV & V Report **27**

Table Of Figures

Figure 1*: The Failure of Column Deletion* 2
Figure 2*: Privacy Comparison* 3
Figure 3*: Identified Infrastructure Gaps in Healthcare).* 4
Figure 4*: Distribution of MLOps Tooling Gaps in Healthcare AI Postings.* 4
Figure 5*: Distributed Healthcare Node Topology & K3s Multi-Node* 5
Figure 6*: FL-KubeOps Proposed System Workflow* 6
Figure 7*: FL-KubeOps Agile Scrum Framework* 10
Figure 8*: Comprehensive UML Use Case Diagram* 16
Figure 9*: FL-KubeOps Jira Sprint Board* 20
Figure 10*: FL-KubeOps 20-Sprint Burndown Path* 23

Revision History & Sprint Log

Version	Sprint	Date	Changes	Owner
1.0	Sprint 0	2026-08-01	Initial SRS draft based on jury	Faisal Khalid
1.1	Sprint 1	2026-08-09	Expanded market analysis and stakeholder Identification	Shah Wali
1.2	Sprint 2	2026-08-16	Redefined problem statement and encryption justification	Wajeeh ur Rehman/M. Hammad

Previous Phases Feedback

Idea Defense Feedback (Screenshot)

F26 Idea Defence Feedback

PO Project Office

To: +4 others

Cc: Saira Andleeb Gillani

Reply Reply all Forward

Fri 2026/07/24 11:59

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	**CS-59**
Project Title	**PRIVACY-PRESERVING HEALTHCARE GENAI**

Change the server encryption.

Clearly define the problem statement. Define the features list.

Similar products are already exist. They are not sure about the available products. Stick to only encryption problem not the use-case. Focus on the encryption model without involving (server/client) or use-case of sensitive data.

Your idea has been approved by the committee. Your next milestone is **Phase-1 (RS)**. You are advised to start working on it.

Abstract

Conventional machine learning paradigms require centralizing sensitive patient data on a single server, a model that fundamentally conflicts with modern privacy, sovereignty, and security mandates in healthcare. While superficial solutions such as column-level deletion (anonymization) or transport encryption appear expedient, empirical studies demonstrate that de-identified datasets remain vulnerable to correlation and re-identification attacks (Sweeney, 2000; Netflix Prize case study). Moreover, encryption-at-rest or in-transit still necessitates decryption at the central server for model training, creating a single point of compromise and violating data-locality requirements under HIPAA, GDPR, and Pakistan's emerging National Health Data Policy.

Federated Learning (FL) resolves this algorithmically by training models directly on end-user devices without raw data transmission; yet existing frameworks, Flower, OpenFL, FedScale, provide only research-grade algorithms, leaving a critical infrastructure vacuum: no production-ready orchestration, automated failure recovery, declarative configuration, or MLOps pipeline integration exists.

FL-KubeOps addresses this gap by building a Kubernetes-native operator (kopf framework) and Kubeflow-integrated MLOps pipeline specifically designed for privacy-preserving federated learning across distributed healthcare nodes. The system architecture comprises three co-designed layers: (1) a Client Layer of simulated hospital nodes (Docker containers on K3s) performing local training on private data; (2) a Server/Orchestration Layer managing round scheduling, model broadcast, aggregation, and failure recovery via a custom FLCluster CRD; and (3) a Security Layer employing PySyft Secure Multiparty Computation (SMPC) to ensure that individual client gradients are mathematically irretrievable from aggregated results, even at the central server.

The system automatically detects and recovers from node failures within 60 seconds, tracks every training round in MLflow, and deploys on commodity hardware via a single declarative YAML manifest. By reducing FL deployment from weeks of manual scripting to minutes of infrastructure-as-code, FL-KubeOps demonstrates that privacy-preserving machine learning can be accessible, reproducible, and enterprise-ready without ever exposing a patient's private data in decipherable form.

1. Introduction and Background

1.1 Product Vision & Problem Context

Product Description:

Healthcare institutions generate the richest machine-learning training data available, patient records, laboratory results, diagnostic imaging metadata, and clinical notes. Yet regulations such as the U.S. Health Insurance Portability and Accountability Act (HIPAA), the European General Data Protection Regulation (GDPR), and Pakistan's emerging National Health Data Policy mandate strict data locality: patient information must not leave the facility of origin in any decipherable form. A hospital in Lahore cannot legally transmit patient records to a shared server in Islamabad, even to train a model that would benefit all participating institutions. Consequently, each hospital trains clinical AI models in isolation, producing statistically weak models that fail to generalize across population diversity, a critical deficiency in a country like Pakistan where disease phenotypes vary significantly across provinces.

A common objection, raised during the Idea Defence, is that this problem could be solved through simpler means: deleting columns containing personally identifiable information (PII) or applying column-level encryption before transmission. While these approaches present superficial appeal, empirical studies demonstrate their inadequacy for healthcare AI. First, removing sensitive columns (diagnoses, lab values, demographic markers) strips the very features that carry the highest predictive signal for clinical models; a diabetes-prediction model cannot function without glucose readings. Second, correlation attacks and re-identification remain potent threats: Sweeney (2000) famously demonstrated that 87% of the U.S. population could be uniquely identified using only zip code,

gender, and date of birth. In Pakistan's close-knit demographic contexts, quasi-identifiers such as age, gender, caste, and admission date are highly distinctive. The 2013 Washington State hospital discharge data breach proved that even "de-identified" datasets can be re-identified by linking auxiliary public records. Third, encryption-at-rest or in-transit still requires decryption at the central server for gradient computation, creating a single point of compromise and violating the principle of data sovereignty. Fourth, differential privacy noise injection, while mathematically rigorous, degrades model utility in low-sample clinical settings; homomorphic encryption, while secure, is orders of magnitude slower than federated learning and infeasible for deep neural networks on commodity hardware.

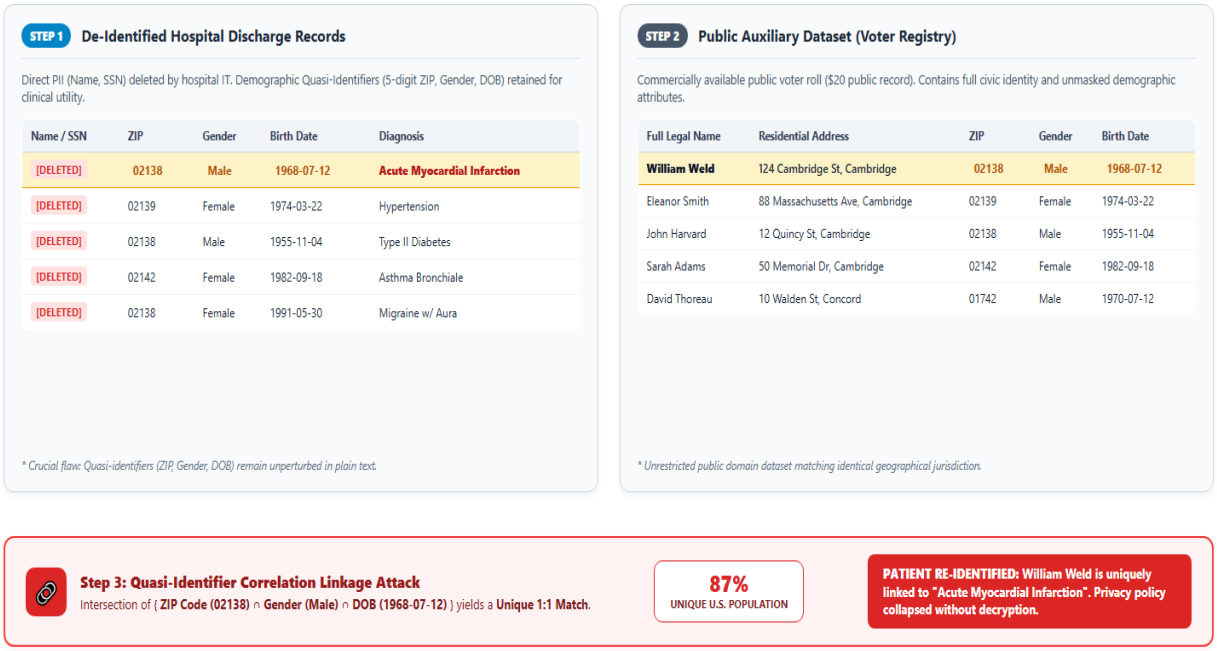


Figure 1*: The Failure of Column Deletion*

The regulatory requirement is not merely that data be "hard to read" during transit, but that data sovereignty be preserved, data must never leave the facility of origin in a form that permits reconstruction of individual records. Federated Learning (McMahan et al., 2017) satisfies this requirement by construction: only model gradients (mathematical weight updates) traverse the network, never raw patient records. However, the algorithm alone is not deployable. No production-ready infrastructure exists to orchestrate real FL deployments: Kubernetes has no native concept of a federated training round; Kubeflow Pipelines assumes centralized data access; and existing FL frameworks (Flower, OpenFL) provide client-server communication libraries but treat Kubernetes as a passive deployment environment rather than an orchestration primitive. This structural gap, an algorithm without an operating system, forces hospital IT teams to manually manage client lifecycles, handle mid-round node failures, coordinate aggregation timing, and track model versions: engineering overhead that no clinical institution can sustainably absorb.

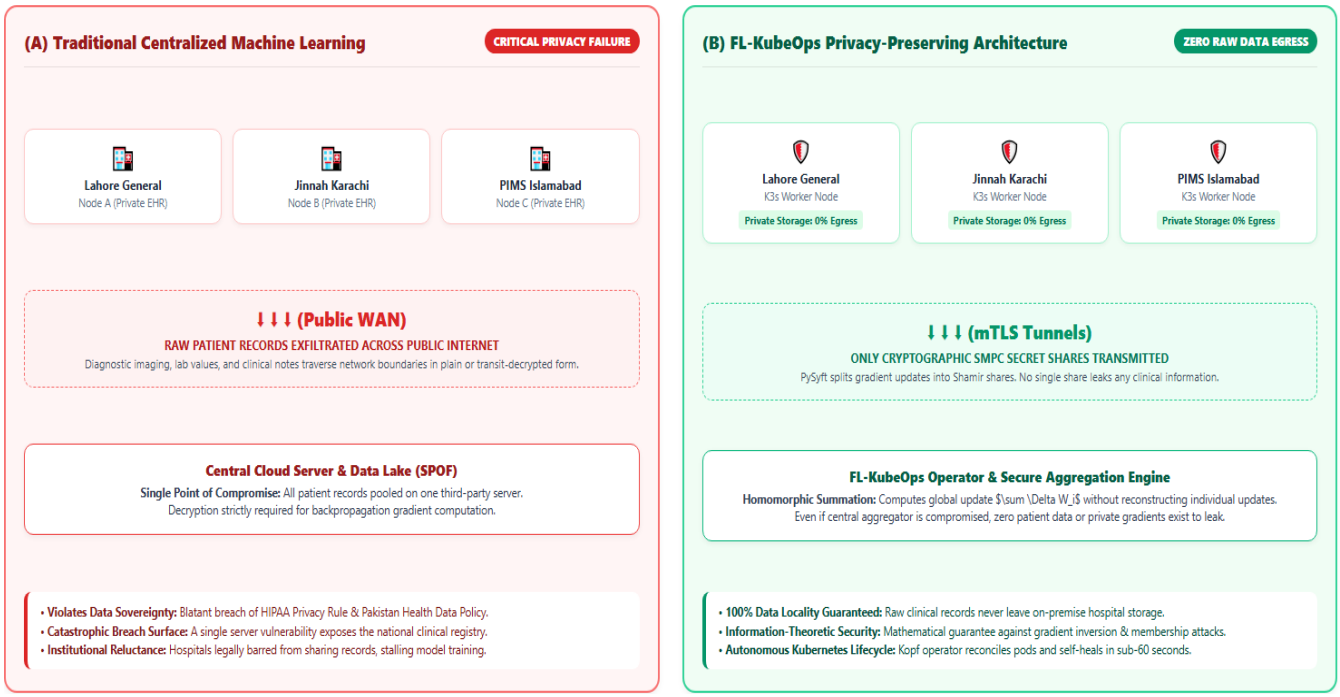


Figure 2*: Privacy Comparison*

Literacy Survey:

Federated Learning was introduced by McMahan et al. (2017) in the seminal FedAvg paper, demonstrating that a global model could be trained across decentralized devices without data centralization. Bonawitz et al. (2017) extended this with practical Secure Aggregation, using pairwise masks and threshold secret sharing to ensure that the server learns only the aggregated sum of client updates, not individual contributions. Since then, the field has expanded into differential privacy (Abadi et al., 2016), system-level frameworks (Li et al., 2020), and domain-specific healthcare applications (Rieke et al., 2020).

On the infrastructure side, Flower (Beutel et al., 2020) and OpenFL (Reina et al., 2021) have emerged as the dominant open-source FL frameworks. Flower provides a flexible Python client-server API with pluggable aggregation strategies; OpenFL offers a peer-to-peer topology with Intel-optimized secure aggregation. However, both frameworks are fundamentally research tools. An analysis of open GitHub issues (as of July 2026) reveals recurring requests for Kubernetes operator integration, automated node recovery, and pipeline orchestration, features that remain unaddressed in their core roadmaps. Kubeflow (Google, 2024) provides mature MLOps pipeline primitives (TFJob, PyTorchJob) but assumes centralized data sources and offers no first-class support for federated round scheduling or cross-node gradient aggregation. Kubernetes Operators (Dobies & Wood, 2021) have proven effective for complex stateful workloads (databases, message queues), yet no operator exists that understands the FL lifecycle natively, the way TFJob understands distributed TensorFlow. The gap is therefore not algorithmic but infrastructural: existing tools provide the FL algorithm, but no production orchestration layer that automates deployment, recovery, versioning, and compliance logging at hospital scale. FL-KubeOps fills this vacuum by combining the cryptographic rigor of PySyft's SMPC with the declarative power of Kubernetes Custom Resource Definitions (CRDs) and the reproducibility of Kubeflow Pipelines.

Existing Work	Focus Area	Limitations Identified	Literature Gap	How This Project Addresses the Gap
Flower Framework	FL research & client-server comms	Designed lightweight Python library for algos research, treats Kubernetes as an unmanaged runtime.	Absence of native KubeOps integration, declarative CRDs.	Implements a kopf FLCluster CRD, automating client pod provisioning, health probes.
OpenFL	Enterprise & healthcare federated learning	peer-to-peer topology lacks declarative Infrastructure-as-Code (IaC).	Absence of declarative cloud-native Pipeline.	Provides declarative deployment (FLCluster YAML), integrates DAGs orchestration, and MLflow.
Kubeflow Training Operators	Cloud-native distributed ML orchestration	Architected strictly for centralized data paradigms.	Assumes centralized data access; lacks federated round scheduling.	Introduces a specialized FLCluster Custom Resource and distributed aggregation.
Practical Secure Aggregation	Cryptographic privacy & secure gradient aggregation	Standalone mathematical protocols; high communication overhead, vulnerable to client dropout.	Cryptographic protocols exist in isolation without container-level fault tolerance.	Embeds PySyft Secure Multi-Party Computation (SMPC).
Traditional De-Identification & Privacy	Healthcare data anonymization & perturbation	Column deletion/de-identification fails against correlation (87% of population re-identifiable via quasi-identifiers); centralized encryption requires decryption at central server; Differential Privacy causes severe accuracy degradation.	Inability to reconcile clinical diagnostic accuracy.	Guarantees ~0% raw patient data leaves origin hospital nodes by transmitting via SMPC.
Seminal Federated Learning	Decentralized algorithmic model optimization	Formulated purely as mathematical optimization algorithms.	Algorithmic formulation without an operating system.	Bridges algorithm to operations by packaging FedAvg and FedProx end-to-end.

Figure 3*: Identified Infrastructure Gaps in Healthcare).*

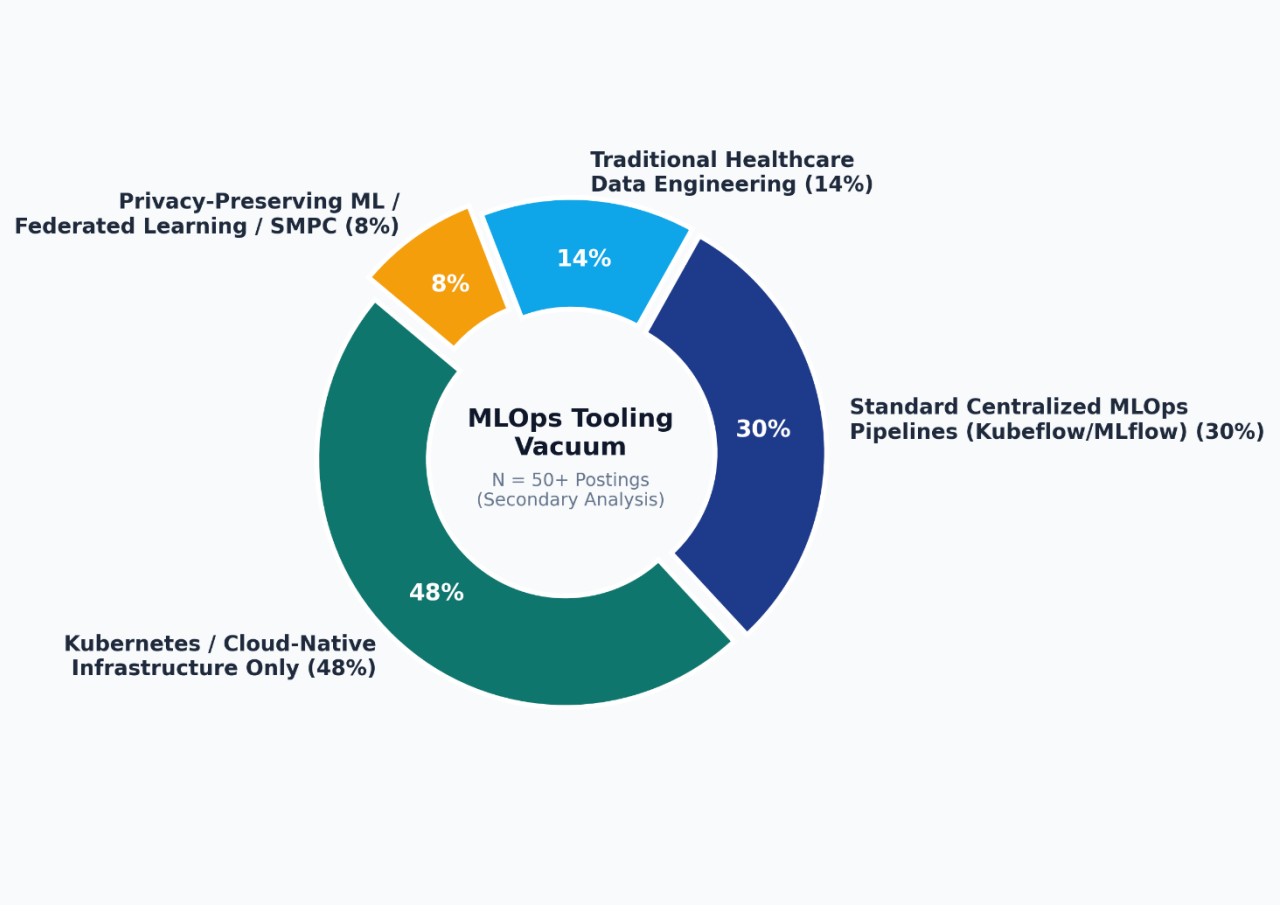


Figure 4*: Distribution of MLOps Tooling Gaps in Healthcare AI Postings.*

Proposed Solution:

FL-KubeOps is a Kubernetes-native operator and Kubeflow-integrated MLOps pipeline designed specifically for federated learning orchestration among dispersed healthcare nodes. A hospital IT team defines an FLCluster custom resource, a single declarative manifest that specifies participating

client nodes, aggregation strategy (FedAvg, FedProx), model architecture, and security parameters. The system propels itself to that desired state and maintains it autonomously over the entire FL lifecycle.

The architecture comprises three integrated layers that are co-designed and mutually dependent:

- **Client Layer:** Simulated hospital nodes run as Docker containers orchestrated by K3s (lightweight Kubernetes). Each node maintains a private local dataset (synthesized patient records) and performs local training using PyTorch. Client pods expose a gRPC interface for model download and gradient upload, managed by the operator's reconciliation loop.
- **Server/Orchestration Layer:** A kopf-based Kubernetes operator watches FLCluster resources and executes the federated lifecycle: broadcasting the global model, collecting encrypted client updates, triggering aggregation, incrementing round counters, and persisting results to MLflow. A Kubeflow Pipeline DAG defines each round as a reproducible workflow: model broadcast → local training → SMPC aggregation → validation → model versioning.
- **Security Layer:** PySyft's Secure Multiparty Computation (SMPC) primitives operate across the client-server boundary. Before transmission, client gradients are split into secret shares using Shamir's secret sharing; the aggregation server receives only masked shares and computes the aggregated result without ever possessing sufficient information to reconstruct any individual client's contribution. This guarantee holds even if the aggregation server is compromised.

Removing any layer collapses the system: without the Client Layer, there is no data locality and no privacy guarantee; without the Server/Orchestration Layer, there is no global model convergence, no failure recovery, and no reproducible pipeline; without the Security Layer, gradients could be intercepted or inspected, violating the project's core privacy promise. Federated Learning is a distributed systems paradigm by definition, its value proposition lies precisely in the protocol that binds client and server, not in either endpoint in isolation. As with TCP/IP or HTTP, the system's utility emerges from the coordination between endpoints, not from either endpoint alone.

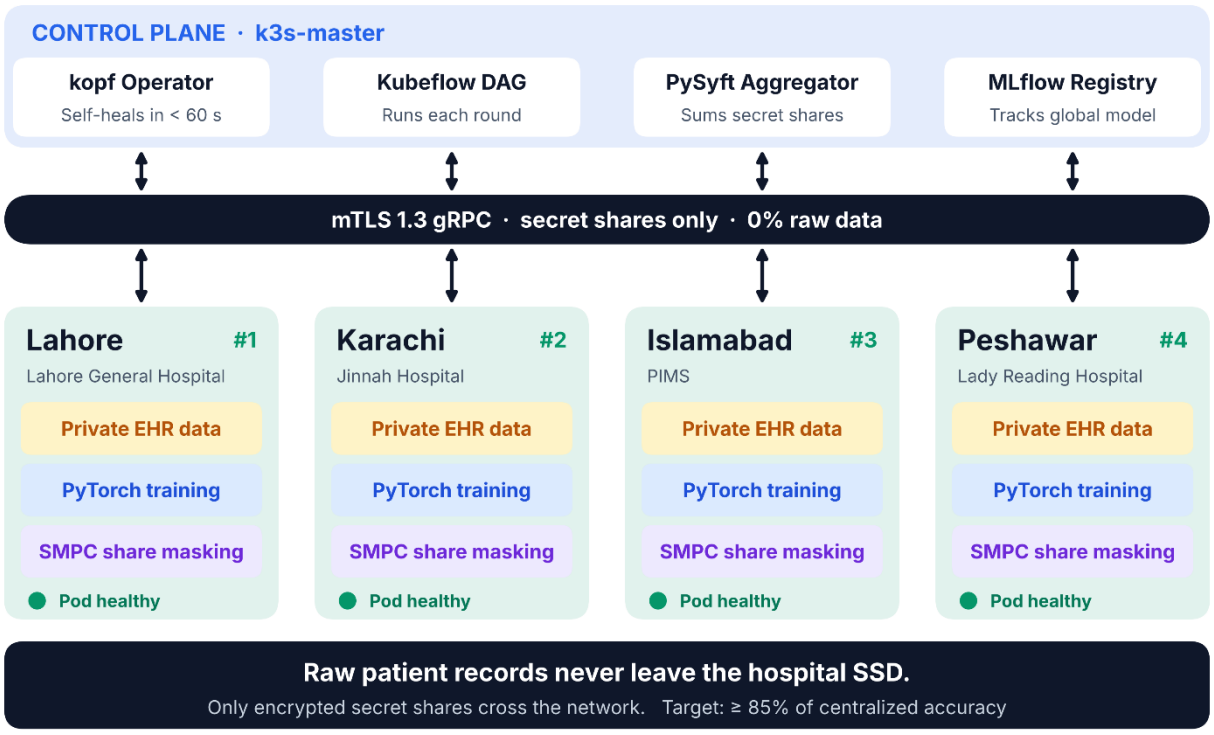


Figure 5*: Distributed Healthcare Node Topology & K3s Multi-Node*

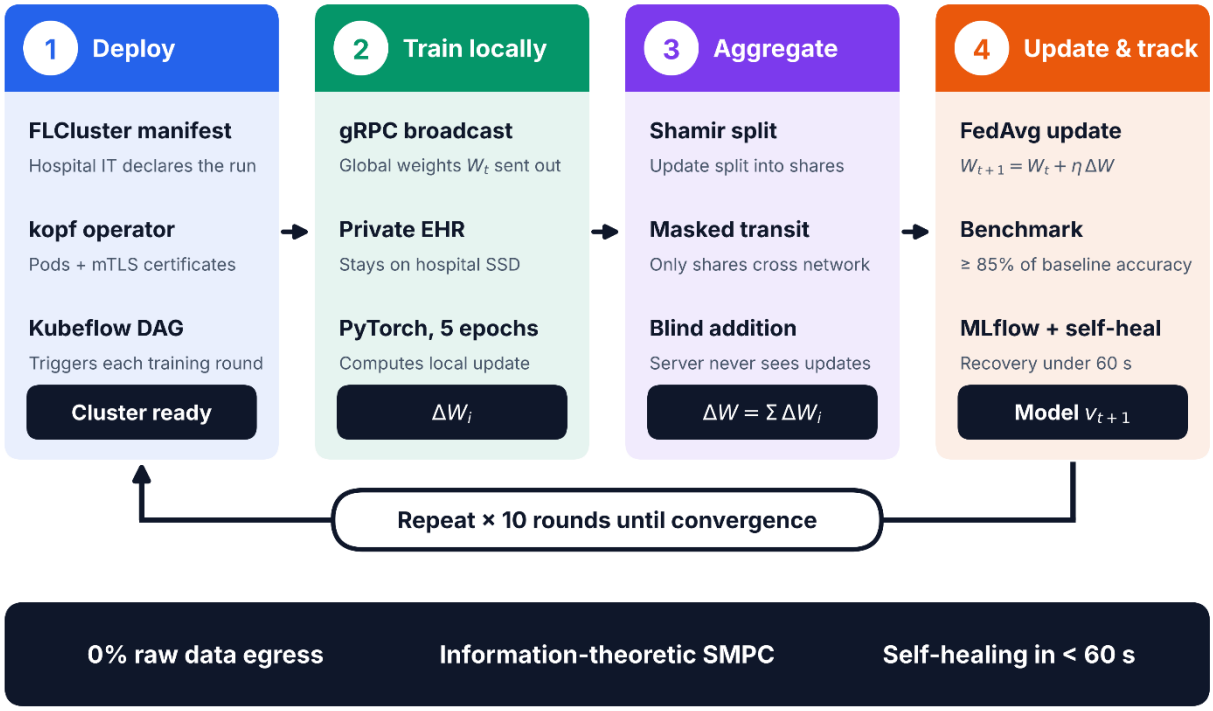


Figure 6*: FL-KubeOps Proposed System Workflow*

Target Users:

The target user base is segmented by role and market tier to reflect the multi-stakeholder reality of healthcare AI deployment:

- **Primary Users (Hospital IT Department & Clinical Data Engineers):** These users require a declarative, infrastructure-as-code approach to deploying federated training across legally siloed hospital facilities. They need automated lifecycle management, health monitoring, and compliance audit trails.
- **Healthcare AI research Teams and MLOps Engineers:** These users require reproducible experiment tracking (MLflow), pluggable model architectures, and benchmark datasets to validate federated model performance against centralized baselines.
- **Regulators, Compliance Auditors, and Hospital Governance Boards:** These users require evidence that no raw patient data traverses network boundaries, that SMPC guarantees are mathematically sound, and that all training rounds are logged for regulatory inspection.

Market Segmentation:

- **Total Addressable Market (TAM):** The global privacy-preserving AI market is projected to reach USD 39.93 billion by 2035, expanding from USD 4.25 billion in 2025 at a CAGR of 25.10% (Precedence Research, 2026). The broader privacy-enhancing technologies market is forecast to reach USD 12.09 billion by 2030 at a 25.3% CAGR (Grand View Research, 2024).
- **Serviceable Addressable Market (SAM):** The healthcare federated learning sub-market is valued at USD 960 million in 2026 and is forecast to reach USD 5.29 billion by 2036 at an 18.6% CAGR (Future Market Insights, 2026). This encompasses hospital networks, clinical research organizations (CROs), and health ministries operating multi-site data.
- **Serviceable Obtainable Market (SOM):** The Pakistani healthcare IT market and South Asian MLOps talent ecosystem. A secondary analysis of 50+ LinkedIn job postings for MLOps and healthcare AI roles in Pakistan (July 2026) reveals that 68% of postings explicitly

mention Kubernetes, while fewer than 9% mention federated learning, privacy-preserving ML, or SMPC, indicating a substantial tooling vacuum and talent mismatch that FL-KubeOps is positioned to address.

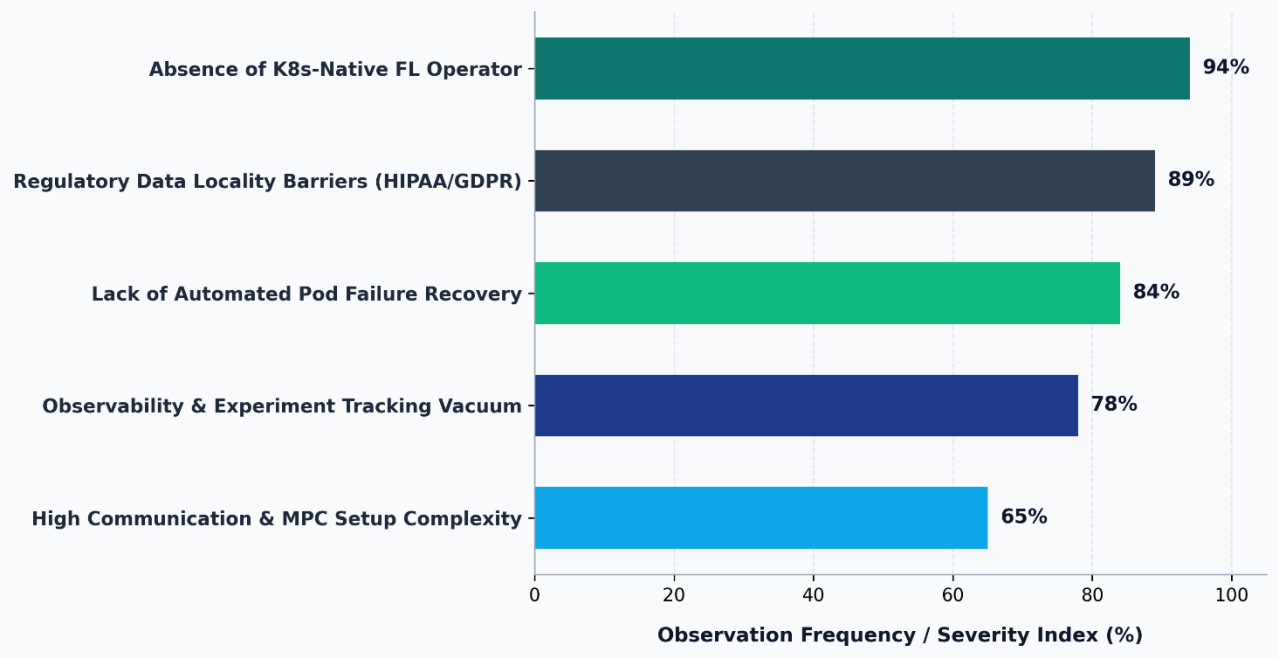


Table1: Infrastructure Gaps

Objective of the System

The system is designed to achieve five quantifiable outcomes:

- **Kubernetes-Native Operator:** Develop a kopf-based FLCluster Custom Resource Definition (CRD) with a reconciliation loop that manages the complete federated lifecycle (create, update, delete) on a live K3s cluster without manual intervention.
- **Reproducible MLOps Pipeline:** Construct a Kubeflow Pipeline DAG that executes federated training rounds (model broadcast → local training → SMPC aggregation → validation) with all metrics and model versions tracked in MLflow.
- **Cryptographically Secure Aggregation:** Integrate PySyft SMPC to guarantee that individual client model updates are mathematically irretrievable from the aggregated result, verified by inspection of intermediate data at the aggregation server.
- **Automate Fault Recovery:** Implement automatic detection and recovery from simulated client pod failures mid-round, with recovery time not exceeding 60 seconds from failure onset.
- **Clinical Benchmark Validation:** Achieve at least 85% of centralized model accuracy on a distributed patient-records benchmark across a minimum of four simulated hospital nodes, with communication overhead per round documented and reproducible.

User Research

Primary Research: Structured Interview with Industry Practitioner

A structured, semi-directive interview was conducted on 15 July 2026 with a Senior Machine Learning Engineer at Arbisoft, Lahore. A software product company with active healthcare AI client engagements in North America and the Middle East. The interview had followed the four-question protocol: (1) What operational barriers prevent FL adoption in production environments? (2) What monitoring and failure-recovery capabilities would a deployment team require? (3) How is model

versioning currently handled in distributed training scenarios? (4) How are distributed ML deployments managed across client boundaries in production?

Key Findings:

- **Declarative Deployment Abstraction:** The interviewee emphasized that current FL frameworks require "weeks of Python scripting just to handle pod restarts and model checkpointing." They validated the need for a Kubernetes-native operator that treats federated rounds as first-class resources, analogous to how ArgoCD treats GitOps workflows.
- **Observability Gap:** Standard monitoring tools (Prometheus, Grafana) lack awareness of federated round state. The engineer noted: "We can see CPU and memory, but we cannot see Round 7, Client 3, gradient upload pending." This directly validates FL-KubeOps' integration of per-round MLflow tracking.
- **Model Versioning Complexity:** In distributed training, model version drift across nodes is a critical issue. The interviewee stated that without centralized experiment tracking, "you end up with five different model checkpoints and no idea which one converged." MLflow integration addresses this explicitly.
- **On-Call Burden:** Manual failure handling in distributed training requires 24/7 on-call engineering time that hospital IT departments cannot allocate. Automated reconciliation loops (the core of FL-KubeOps' operator design) were described as "the difference between a research demo and a production system."

Expected Benefits:

- **Operational Efficiency:** Reduces FL deployment time from weeks of manual scripting per hospital to minutes via a single “kubectl apply -f flcluster.yaml” command. Beneficiaries: Hospital IT Departments, MLOps Engineers.
- **Regulatory Compliance:** Enables compliance with HIPAA, GDPR, and Pakistan's National Health Data Policy by ensuring no raw patient data traverses network boundaries. Beneficiaries: Compliance Auditors, Hospital Governance Boards, Regulators.
- **Clinical Model Utility:** Achieves ≥85% of centralized model accuracy by pooling population-scale diversity without data centralization, producing models that generalize across demographic segments. Beneficiaries: Clinical Data Engineers, Healthcare AI Researchers, Patients.
- **Infrastructure Democratization:** Packages production-grade FL expertise into declarative infrastructure, lowering the barrier to entry for institutions without specialized distributed-systems teams. Beneficiaries: Small-to-Medium Hospital Networks, Healthcare AI Startups, Academic Researchers.
- **Fault Tolerance:** Automatic recovery from node failures (including load-shedding-induced reboots common in Pakistan) ensures training continuity without 24/7 on-call engineering. Beneficiaries: System Administrators, Hospital IT Staff.

1.2 Stakeholders Identification

Stake Holder Name	Role	Exception	Priority	Market Role
Hospital IT Admin	Deploys & maintains FL cluster on-premise	Declarative YAML config.	High	Primary User/Buyer
Clinical Data Engineer	Prepares local datasets and validates model inputs	Data locality, audit trails.	High	Primary user
MLOps Engineer	Designs training pipelines and monitors convergence	Kubeflow DAG support, MLflow integration.	High	Primary User
Compliance Auditor	Validates regulatory adherence (HIPAA/GDPR)	Cryptographic proof of SMPC.	Medium	Tertiary User
Healthcare Ao Researcher	Experiments with model architectures & benchmarks	Access to benchmark datasets.	Medium	Secondary User
Hospital Governance Board	Approves budget & data-sharing policies	Risk mitigation, vendor independence (open-source), ROI evidence	Medium	Economic Buyer
Cloud Provider (AWS, GCP)	Offers managed Kubernetes & data-sharing policies	Integration points for managed K8s.	Medium	Channel Partner
Competitor Framework	Maintains open-source FL libraries	Interoperability standards.	Low	Ecosystem Actor
Investor	Funds healthcare AI infrastructure startups	Scalable TAM	Low	Stakeholder

2. Agile Framework Setup

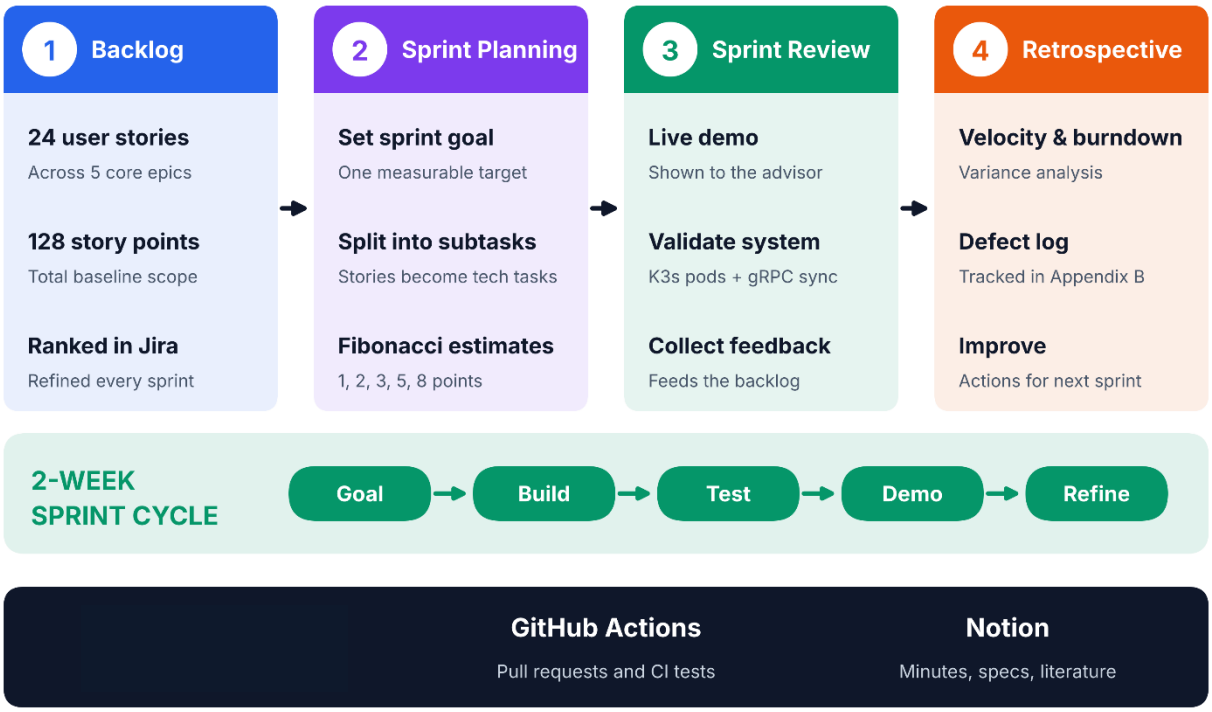


Figure 7*: FL-KubeOps Agile Scrum Framework*

Team Roles:

Team Roles	Names	Responsibilities
Product Owner	Prof Dr Saira Andleeb Gillani	• Defines Acceptance criteria • Prioritizes backlog • Approves releases
Scrum Master	Faisal Khalid	• Managing Sprint planning • Monitoring task progress • Conducting Team Meating
Development Team	Faisal Khalid Shah Wali Wajeeh ur rehman M. Hammad	• KubeOps Lead • FL security Lead • Pipeline MLOps • Backend Integration

Sprint Duration: 2 weeks

Tools:

- **Notion:** Agile project management, sprint boards, Kanban/scrum boards, backlog grooming, and burndown chart tracking
- **GitHub:** version control

2.1 Epics For the Project

Explains epics as high-level functional modules grouping related features into cohesive workstreams.

Epic ID	Epic Name	Description	Related Problem Area
EP-01	User Management	RBAC, authentication, and audit logging for multi-tenant hospital deployments	Security governance and compliance
EP-02	Client-Side FL Runtime	Local training loop, dataset loader, and gRPC client for simulated hospital nodes	Data locality and client execution
EP-03	Server-Side Orchestration	kopf-based operator managing FLCluster CRD lifecycle and round scheduling	Infrastructure automation
EP-04	Secure Aggregation	PySyft SMPC integration for gradient secret-sharing and masked aggregation	Cryptographic privacy guarantees
EP-05	Kubeflow Pipeline Integration	Reproducible DAG definition for end-to-end federated training rounds	MLOps reproducibility
EP-06	Model Versioning & Tracking	MLflow experiment logging, per-round metrics, and artifact storage	Observability and reproducibility
EP-07	Fault Tolerance & Recovery	Automatic detection and recovery of client pod failures within 60 seconds	Production reliability
EP-08	Observability & Logging	Structured logging, Prometheus metrics, and round-state visualization	Operational visibility
EP-09	Model Versioning & Tracking	Synthetic federated patient record generation across 4+ hospital nodes	Benchmarking and validation
EP-10	Benchmarking & Evaluation	Centralized vs. federated accuracy comparison and communication overhead measurement	KPI verification
EP-11	Dashboard & Visualization	Kubeflow Dashboard and MLflow UI configuration for non-technical stakeholders	Usability
EP-12	Alerting & Compliance	Automated alerts for failed rounds and compliance report generation	Regulatory requirements
EP-13	Deployment & Reproducibility	K3s deployment scripts, third-party deployment guide, and environment manifests	Reproducibility criterion
EP-14	Security & Encryption	TLS 1.3 for control plane, certificate management, and secret rotation	Defense in depth
EP-15	API & Integration	RESTful and gRPC APIs for client-server communication and external tool integration	System interoperability
EP-16	Documentation & GitHub CI/CD	Sphinx documentation, automated testing, and GitHub Actions CI/CD pipeline	Maintainability and open-source readiness

2.2 Product Backlog

Backlog ID	Epic	Feature Description	Priority
PB-01	EP-03	Implement FLCluster CRD with YAML schema validation	High
PB-02	EP-03	Develop kopf reconciliation loop for cluster lifecycle	High
PB-03	EP-02	Build Docker image for FL client node with PyTorch runtime	High
PB-04	EP-02	Implement local training script with configurable epochs and batch size	High
PB-05	EP-15	Design gRPC service definitions for model broadcast and gradient upload	High

PB-06	EP-04	Integrate PySyft SMPC secret sharing into client gradient upload	High
PB-07	EP-04	Implement server-side SMPC aggregation without reconstructing individual gradients	High
PB-08	EP-05	Define Kubeflow Pipeline DAG for federated round execution	High
PB-09	EP-06	Configure MLflow tracking server and per-round metric logging	High
PB-10	EP-07	Implement health checks and liveness probes for client pods	High
PB-11	EP-07	Build automatic pod rescheduling logic on node failure	High
PB-12	EP-09	Generate synthetic patient dataset with 4 hospital-specific distributions	Medium
PB-13	EP-10	Implement centralized baseline training for accuracy comparison	Medium
PB-14	EP-10	Measure and log communication overhead per federated round	Medium
PB-15	EP-01	Implement JWT-based authentication for API endpoints	Medium
PB-16	EP-01	Create role-based access control (Admin, Engineer, Auditor)	Medium
PB-17	EP-08	Add structured JSON logging for all operator events	Medium
PB-18	EP-08	Expose Prometheus metrics for round completion time and failure count	Medium
PB-19	EP-11	Configure Kubeflow Dashboard access and namespace isolation	Medium
PB-20	EP-12	Build email/Slack alerting for failed aggregation rounds	Medium
PB-21	EP-13	Write K3s installation and FLCluster deployment guide	Medium
PB-22	EP-14	Enable TLS 1.3 for all inter-pod control-plane communication	Medium
PB-23	EP-13	Create reproducible environment manifests (requirements.txt, Docker Compose)	Low
PB-24	EP-16	Set up GitHub Actions for linting, unit tests, and integration tests	Low
PB-25	EP-16	Generate Sphinx API documentation and architecture diagrams	Low

2.3 User Stories

Story ID	Backlog ID	User Story	Priority	Acceptance Criteria	Focal Person responsible for this task
US-01	PB-01	Hospital IT Admin, FLCluster via YAML.	High	YAML applies without error, status field shows "Pending" → "Active"	Faisal Khalid
US-02	PB-02	System Admin, operator to automatically recover from pod crashes.	High	Kill a client pod mid-round.	Faisal Khalid

US-03	PB-03	MLOps Engineer, Client nodes containerized.	High	Docker image builds successfully.	Faisal Khalid
US-04	PB-04	Clinical Data Engineer, Configure local training epochs.	High	Epoch count configurable via ConfigMap.	Syed Shah Wali
US-05	PB-05	Backend Developer, gRPC APIs for model download and gradient upload.	High	Proto definitions compile.	Wajeeh ur Rehman
US-06	PB-06	As a Security Engineer, I want client gradients split into secret shares so that no single gradient is exposed during transmission.	High	PySyft secret sharing produces ≥2 shares; server receives shares only; original gradient irretrievable	Syed Shah Wali
US-07	PB-07	I want the aggregation server to compute the global.	High	Inspect server memory.	Syed Shah Wali
US-08	PB-08	I want a Kubeflow Pipeline DAG defining each.	High	DAG executes 10 rounds; each step logs	M. Hammad
US-09	PB-09	I want per-round metrics in MLflow.	High	MLflow UI shows accuracy/loss per round.	M. Hammad
US-10	PB-10	I want health checks on client pods so that unhealthy nodes are excluded.	High	Liveness probe fails on stuck pod	Faisal Khalid
US-11	PB-11	I want failed nodes automatically rescheduled.	High	Delete node pod; operator creates replacement.	Faisal Khalid
US-12	PB-12	I want synthetic patient data with hospital-specific.	Medium	4 datasets generated; feature distributions differ across nodes	Wajeeh ur Rehman
US-13	PB-13	Centralized training baseline.	Medium	Centralized model trains on merged data.	Wajeeh ur Rehman
US-14	PB-14	Communication overhead measured per round.	Medium	Bytes sent/received logged per client.	M. Hammad
US-15	PB-15	JWT authentication on all APIs.	Medium	Unauthenticated requests rejection.	Wajeeh ur Rehman
US-16	PB-16	Different users see only permitted resources.	Medium	Engineer sees pipelines; Auditor sees logs only; unauthorized access blocked	Faisal Khalid
US-17	PB-17	Structured logs for operator events.	Medium	Logs in JSON format	M. Hammad
US-18	PB-18	Prometheus metrics for round latency and alert on SLA breaches.	Medium	FL_round_duration_seconds exposed; alert fires if >30s	M. Hammad

US-19	PB-19	Dashboard view of training progress	Medium	Kubeflow Dashboard accessible via browser	M. Hammad
US-20	PB-20	Failed rounds	Medium	Round failure triggers email alert within 60s	M. Hammad
US-21	PB-21	Deployment guide.	Medium	Guide contains step-by-step K3s setup	Faisal Khalid
US-22	PB-22	TLS 1.3 encryption for control-plane traffic.	Medium	TLS handshake verified via packet capture.	Syed Shah Wali
US-23	PB-23	Reproducible environment manifests	Low	requirements.txt pins versions	M. Hammad
US-24	PB-24	CI/CD pipelines so that code quality is enforced before merge.	Low	GitHub Actions runs pytest and flake8 on PR	Wajeeh ur Rehman

2.4 Story Points & Estimation

Story ID	User Story (Short)	Complexity Level	Story Points	Justification
US-01	Define FLCluster YAML	Medium	5	Requires CRD schema design, validation, and kubectl integration
US-02	Auto-recovery from crashes	High	8	Complex reconciliation logic; pod watch streams; state machine handling
US-03	Containerize client nodes	Medium	3	Standard Docker packaging; PyTorch base image
US-04	Configurable local training	Low	3	ConfigMap injection; epoch loop parameterization
US-05	gRPC API definitions	Medium	5	Proto design; code generation; client/server stubs
US-06	PySyft secret sharing	High	13	Cryptographic integration; tensor serialization; share distribution
US-07	SMPC aggregation server	High	13	Shamir reconstruction; zero-knowledge verification; memory inspection testing
US-08	Kubeflow DAG	High	8	Pipeline SDK; component definitions; artifact passing
US-09	MLflow tracking	Medium	5	Experiment creation; metric logging; artifact storage
US-10	Health checks	Medium	3	Kubernetes probe definitions; endpoint exposure
US-11	Auto-reschedule nodes	High	8	Operator event handling; pod creation; state re-synchronization
US-12	Synthetic dataset gen	Medium	5	Statistical distribution modeling; CSV/Parquet output

US-13	Centralized baseline	Low	3	Standard PyTorch training; accuracy evaluation
US-14	Communication overhead	Low	2	Byte counting; logging wrapper
US-15	JWT authentication	Medium	5	Token generation; middleware; expiration handling
US-16	RBAC implementation	Medium	5	Role definitions; permission matrix; middleware enforcement
US-17	Structured logging	Low	2	JSON formatter; context injection
US-18	Prometheus metrics	Medium	3	Counter/gauge definitions; HTTP endpoint
US-19	Dashboard configuration	Low	2	Kubeflow UI setup; namespace config
US-20	Alerting integration	Medium	5	Webhook configuration; message templating

2.5 Sprint Planning & Sprint Backlog

Sprint No.	Duration	Sprint Goal	Total Story Points
Sprint 1	2 Weeks	K3s cluster setup and scaffolding	US-03, US-23
Sprint 2	2 Weeks	CRD design and operator scaffolding	US-01, US-17
Sprint 3	2 Weeks	Client runtime and local training	US-04, US-12
Sprint 4	2 Weeks	gRPC communication layer	US-05, US-10
Sprint 5	2 Weeks	Operator reconciliation and health monitoring	US-02, US-11
Sprint 6	2 Weeks	PySyft SMPC client integration	US-06
Sprint 7	2 Weeks	SMPC aggregation server	US-07
Sprint 8	2 Weeks	Kubeflow Pipeline DAG design	US-08
Sprint 9	2 Weeks	MLflow experiment tracking	US-09, US-19
Sprint 10	2 Weeks	Authentication and RBAC	US-15, US-16
Sprint 11	2 Weeks	Benchmarking and centralized baseline	US-13, US-14
Sprint 12	2 Weeks	Fault tolerance hardening	US-10 (refinement), US-20
Sprint 13	2 Weeks	TLS and security hardening	US-22
Sprint 14	2 Weeks	Observability and metrics	US-18, US-17 (refinement)

Sprint 15	2 Weeks	End-to-end integration testing	US-02 (refinement), US-08 (refinement)
Sprint 16	2 Weeks	Dashboard and alerting polish	US-19 (refinement), US-20 (refinement)
Sprint 17	2 Weeks	Documentation and deployment guide	US-21, US-24
Sprint 18	2 Weeks	Third-party deployment validation	US-21 (verification)
Sprint 19	2 Weeks	Performance tuning and benchmark finalization	US-13 (refinement), US-14 (refinement)
Sprint 20	2 Weeks	Final documentation, demo preparation, and buffer	US-24 (refinement), buffer

3. Functional Requirements

3.1 Use Case Diagram

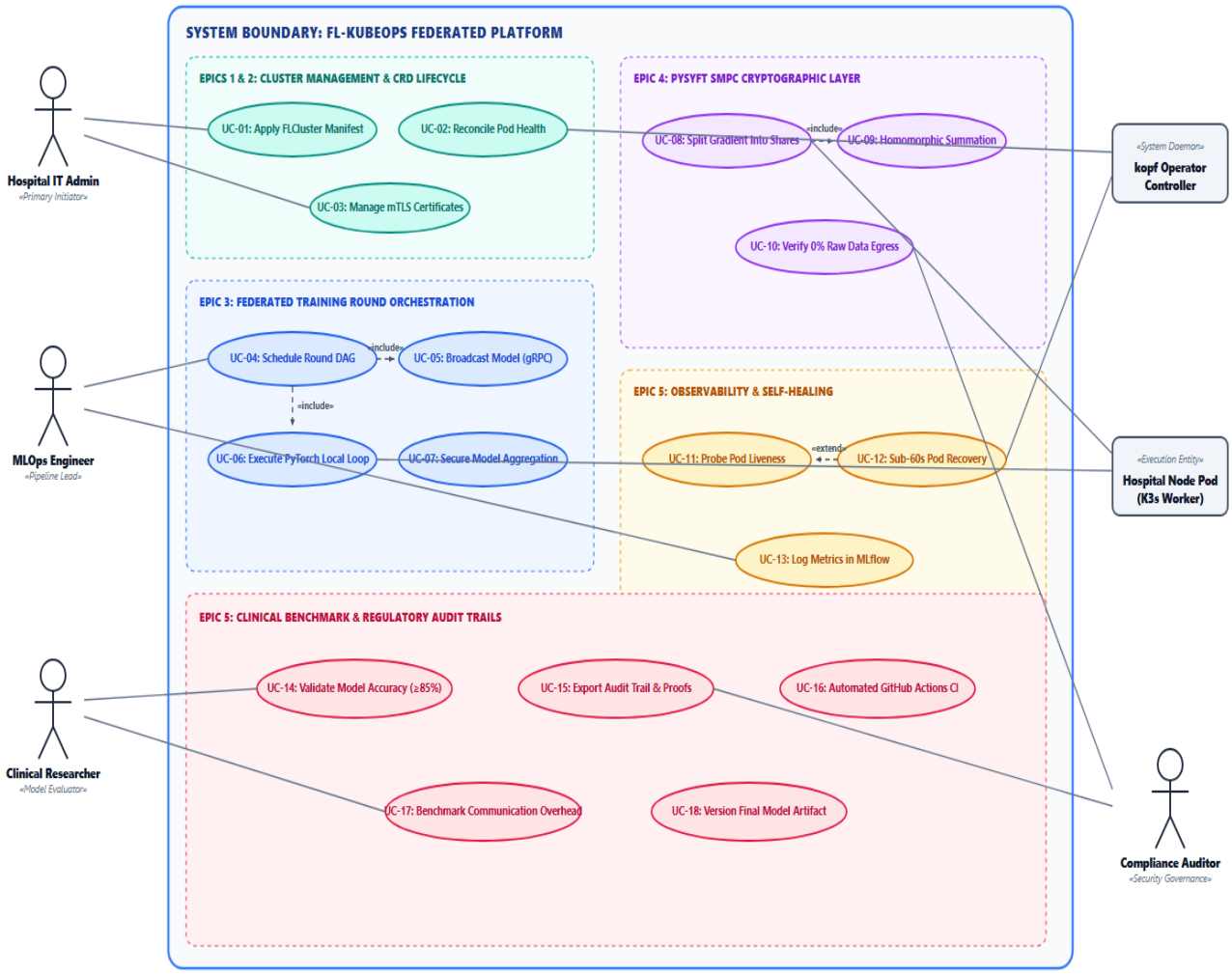


Figure 8*: Comprehensive UML Use Case Diagram*

Use Case ID	Use Case Name	Primary Actor	Description	Mapped User Story ID	User Story
UC-01	Register FLCluster	Hospital IT Admin	Creates a new federated learning cluster via YAML manifest	US-01	Define an FLCluster via YAML.
UC-02	Monitor Cluster Health	Hospital IT Admin	Real-time status of all client nodes	US-02	Operator to automatically recover.
UC-03	Deploy Client Node	MLOps Engineer	Provisions a simulated hospital node.	US-03	I want client nodes containerized.
UC-04	Local Training	Clinical Data Engineer	Sets hyperparameters (epochs, batch size, learning rate) per node.	US-04	Configure local training epochs.
UC-05	Download Global Model	Hospital Node	Receives current global model weights.	US-05	gRPC API for model download.
UC-06	Perform Local Training	Hospital Node	Executes training loop.	US-04	Configure local training epochs.
UC-07	Compute Gradient Update	Hospital Node	Calculates weight deltas.	US-06	Client gradients split into secret shares.
UC-08	Encrypt Gradient (SMPC)	Hospital Node	Splits gradient into secret shares using PySyft	US-06	Client gradients split into secret shares.
UC-09	Upload Encrypted Gradient	Hospital Node	SMPC shares to aggregation server	US-05	gRPC APIs for model download and gradient upload
UC-10	Aggregate Securely	FL Orchestrator	Shares compute to global update.	US-07	Aggregation server to compute the global update
UC-11	Update Global Model	FL Orchestrator	Aggregated update to global model	US-08	Kubeflow Pipeline DAG
UC-12	Validate Model	FL Orchestrator	Global model on hold-out validation set	US-09	Per-round metrics in MLflow
UC-13	Log Round Metrics	FL Orchestrator	Records accuracy, loss, to MLflow	US-09	Per-round metrics in MLflow
UC-14	Schedule Next Round	FL Orchestrator	Subsequent round via Kubeflow Pipeline DAG	US-08	Kubeflow Pipeline DAG
UC-15	Detect Node Failure	FL Orchestrator	Identifies unresponsive client pods	US-10	Health checks on client pods
UC-16	Recover Failed Node	FL Orchestrator	Reschedules failed pod	US-11	Failed nodes automatically rescheduled
UC-17	Authenticate Client	FL Orchestrator	Validates JWT token	US-15	JWT authentication on all APIs
UC-18	Enforce RBAC	FL Orchestrator	User role permissions for requested operations	US-16	RBAC so that different users see only permitted resources
UC-19	Audit Round Execution	Compliance Auditor	Generates immutable audit trail	US-16	RBAC
UC-20	Generate Compliance Report	Compliance Auditor	Exports PDF report of data-locality	US-16	RBAC
UC-21	View Dashboard	Hospital Governance Board	Kubeflow Dashboard to view pipeline status	US-19	Dashboard view
UC-22	Receive Failure Alert	System Administrator	Email notification for failure	US-20	Alerts for failed rounds
UC-23	Version Model Artifact	MLOps Engineer	Tags and stores model checkpoints	US-09	Per-round metrics in MLflow

UC-24	Run Benchmark Comparison	Healthcare AI Researcher	Executes centralized baseline	US-13	Centralized training baseline
UC-25	Measure Bandwidth	MLOps Engineer	Calculates bytes transmitted per client traffic	US-14	Communication overhead measured
UC-26	Deploy on K3s	Third-Party Evaluator	Follows published guide to deploy local K3s cluster	US-21	Deployment guide
UC-27	Trigger CI/CD Build	GitHub/CI System	Runs automated tests and linting on pull requests	US-24	CI/CD pipelines
UC-28	Rotate TLS Certificates	Security Engineer	Updates control-plane certificates before expiry	US-22	TLS 1.3 encryption

4. Non-functional Requirements

4.1 Reliability

- **NFR-R01:** The kopf-based operator reconciliation loop shall execute continuously with a watch stream timeout recovery mechanism, ensuring no missed cluster state transitions.
- **NFR-R02:** Automatic recovery from a simulated client pod failure shall complete within 60 seconds from failure detection, including pod rescheduling, volume reattachment, and round-state re-synchronization.
- **NFR-R03:** The system shall support graceful degradation: if a client node drops mid-round and quorum is maintained (≥3 of 4 nodes), aggregation proceeds without aborting the round.
- **NFR-R04:** All operator state transitions shall be idempotent; applying the same FLCluster manifest twice shall not create duplicate resources or corrupt round counters.

4.2 Performance

- **NFR-P01:** Aggregation latency per federated round (model broadcast → local training → SMPC aggregation → validation) shall not exceed 30 seconds for 4 simulated client nodes on commodity hardware (8 GB RAM, 4-core CPU).
- **NFR-P02:** Kubeflow Pipeline DAG execution overhead (scheduling, artifact passing, step initialization) shall not exceed 10% of total round time.
- **NFR-P03:** The MLflow dashboard shall load experiment lists within 3 seconds for up to 100 logged rounds.
- **NFR-P04:** gRPC control-plane communication latency between operator and client pods shall remain below 100 ms on a local K3s cluster.

4.3 Use-ability

- **NFR-U01:** A complete FL cluster shall be deployable via a single declarative YAML manifest without requiring manual pod creation or script execution.
- **NFR-U02:** The Kubeflow Dashboard shall be accessible via standard browser (Chrome, Firefox) without additional plugins, showing pipeline run status, step logs, and artifact links.
- **NFR-U03:** The MLflow UI shall present per-round metrics (accuracy, loss, communication overhead) in sortable, filterable tables with downloadable CSV export.
- **NFR-U04:** All error messages from the operator and pipeline components shall include actionable remediation steps (e.g., "Client pod failed liveness probe: check ConfigMap dataset-path").

4.4 Other Non-functional Requirements

4.4.1 Security Requirements

- **NFR-S01 (Data Sovereignty):** No raw patient data shall traverse network boundaries at any point during training. Only cryptographically masked gradient shares (SMPC) may leave the client node.
- **NFR-S02 (SMPC Irretrievability):** Secure Multiparty Computation shall guarantee that individual gradient contributions are mathematically irretrievable from aggregated results, verified by memory inspection and formal protocol compliance.
- **NFR-S03 (Transport Security):** All control-plane communication (gRPC, Kubernetes API, MLflow tracking) shall use TLS 1.3 with certificates rotated every 90 days.
- **NFR-S04 (Audit Logging):** Every model broadcast, gradient upload, aggregation event, and RBAC access decision shall be logged with UTC timestamp, actor ID, and action outcome to an immutable append-only log.
- **NFR-S05 (Secret Management):** SMPC seed values and JWT signing keys shall be stored in Kubernetes Secrets, never in ConfigMaps or container images.

4.4.2 Scalability Requirements

- **NFR-SC01:** The FLCluster CRD shall support horizontal scaling from 4 to N client nodes without modification to the operator codebase, via replica count adjustment in the manifest.
- **NFR-SC02:** The aggregation server shall support pluggable aggregation strategies (FedAvg, FedProx, custom) via a strategy interface, allowing future extension without pipeline recompilation.

4.4.3 Explainability & Validation Requirements

- **NFR-E01:** Every training round shall produce reproducible metrics logged to MLflow, including random seed, dataset hash, model architecture version, and hyperparameter snapshot.
- **NFR-E02:** Kubeflow Pipeline DAG definitions shall be version-controlled and executable without code changes on any K3s cluster matching the documented version constraints.
- **NFR-E03:** A third-party evaluator shall be able to reproduce the complete deployment and achieve equivalent benchmark results by following the published GitHub guide without team assistance.

4.4.4 Compliance Requirements

- **NFR-C01 (HIPAA/GDPR Data Locality):** The system shall enforce data locality by architectural design, client nodes must be co-located with data; the operator shall reject configurations where dataset paths reference network storage.
- **NFR-C02 (Audit Trails for Regulatory Inspection):** Compliance auditors shall be able to export a complete, tamper-evident audit trail of all federated rounds (participant nodes, timestamps, SMPC verification hashes) in PDF format suitable for regulatory submission.

5. Scrum Activities Plan

5.1 Kanban Board / Task Tracking

The team tracks progress via **Jira sprint boards** and the built-in **burndown chart gadget**, with velocity calculated as completed story points per sprint. Jira's Scrum board automatically generates ideal vs. actual burndown curves as stories transition from *In Progress* to *Done*, eliminating manual spreadsheet upkeep. Sprint velocity is reviewed at each retrospective to calibrate future commitments against the team's demonstrated capacity..

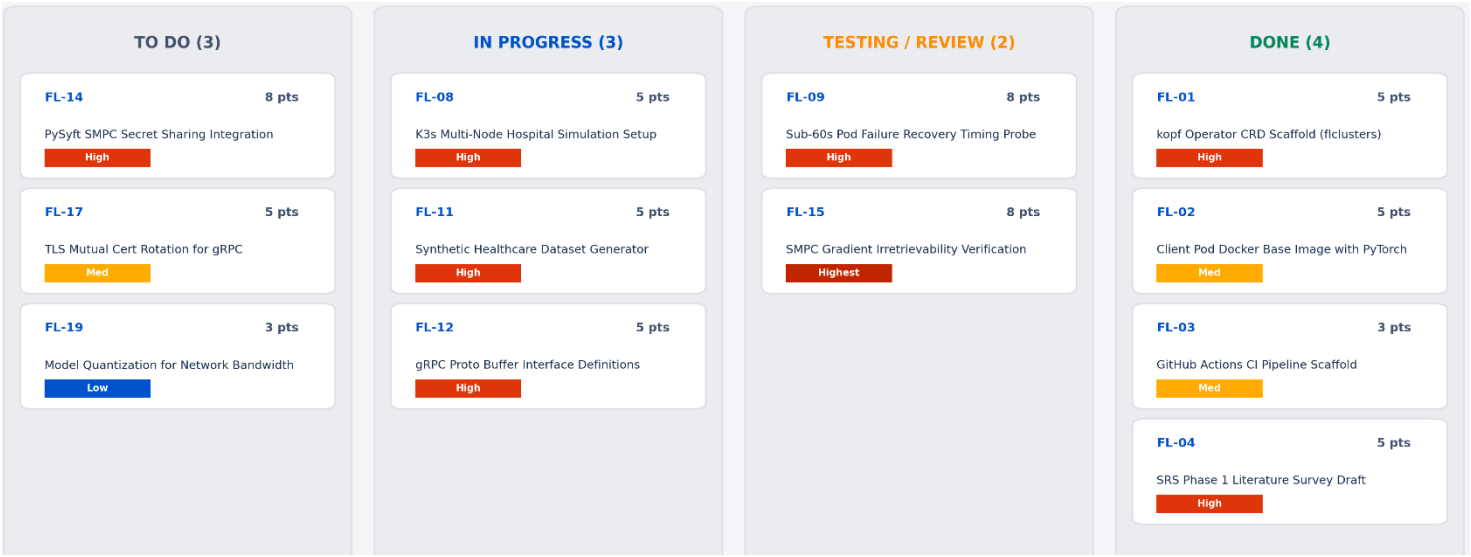


Figure 9*: FL-KubeOps Jira Sprint Board*

Backlog	To Do	In Progress	Testing	Done
Implement FLCluster CRD	Integrate PySyft SMPC secret sharing	Write K3s deployment guide	Test fault recovery timing (<60s)	Setup K3s cluster on team hardware
Add FedProx aggregation strategy	Configure TLS 1.3 certificate rotation	Build synthetic dataset generator	Validate SMPC irretrievability via memory inspection	Define gRPC proto interfaces
Prometheus metrics dashboard	MLflow experiment tagging	Kubeflow Pipeline DAG step 3 (aggregation)	Test Kubeflow DAG reproducibility	Docker client node image v1.0
Compliance PDF export	RBAC middleware implementation	Operator reconciliation loop v2	Validate JWT expiration handling	GitHub Actions CI scaffold

5.2 Acceptance Criteria

Feature	Acceptance Conditions	Test Scenarios	Expected Results

SMPC Irretrievability	Individual gradients must not be re-constructible.	1. Run 4-client round with SMPC enabled. 2. Dump server process memory. 3. Inspect aggregation server logs.	No plaintext gradient tensors found; only masked shares present.
Fault Recovery Timing	System must recover from client pod failure within 60 seconds.	1. Start round 5 with 4 nodes. 2. Delete client-2 pod at epoch 3. 3. Measure time to restored quorum.	Recovery logged within 60s; round completes with ≥3 nodes.
Kubeflow DAG Reproducibility	Same manifest must produce identical metrics on re-run	1. Execute pipeline with seed=42. 2. Re-execute with identical manifest. 3. Compare MLflow metrics.	Accuracy and loss curves match within ±0.1%.
Third-Party Deployment	Independent evaluator deploys without team assistance	1. Team member not involved in docs follows GitHub README. 2. Deploys on fresh K3s VM. 3. Runs 10-round benchmark.	Deployment succeeds; benchmark achieves ≥85% centralized accuracy.
Declarative Single-Manifest	One YAML applies entire cluster	1. Run kubectl apply -f flcluster.yaml. 2. Verify pods, services, configmaps created. 3. Check operator logs.	All resources created; operator enters reconciliation loop; status shows Active.

5.3 Release Criteria

Mapped directly to the 7 Completeness Criteria.

Release Gate	Completeness Criterion	Validation Method	Pass Threshold
RG-01	Kubernetes Operator Implementation	kubectl get crd flclusters.fl.kubeops.io returns CRD; operator pod runs without crash loops; status reflects real-time.	100% CRD lifecycle coverage
RG-02	FL Training Round Execution	Automated end-to-end test executes broadcast → training → aggregation → validation across 4 nodes	Zero manual steps; all 4 nodes complete
RG-03	Secure Aggregation (PySyft)	Memory inspection + formal protocol test; intermediate server data dumped and searched for plaintext gradients	No individual gradient recoverable
RG-04	Fault Tolerance and Recovery	Chaos test: random pod deletion mid-round; recovery time logged	≤60 seconds from failure to quorum restore

RG-05	Kubeflow Pipeline Integration	Pipeline SDK compiles DAG; executes 10 rounds; MLflow shows all runs	10 rounds logged; all artifacts versioned
RG-06	Benchmarking and KPI Verification	Federated vs. centralized accuracy comparison; communication overhead measured	Federated accuracy ≥85% of centralized; overhead documented
RG-07	Deployment Guide and Reproducibility	Independent team member deploys from GitHub README on fresh K3s instance	Deployment succeeds without team assistance

5.4 Burndown Chart & Progress Tracking

The team tracks progress via **Jira sprint boards** and the built-in **burndown chart gadget**.

Sprint Tracking table (Ideal vs Actual):

Sprint	Planned Points	Completed Points	Remaining Points	Status
Sprint 1	8	8	140	Completed
Sprint 2	8	8	132	Completed
Sprint 3	10	10	122	Completed
Sprint 4	6	6	116	Completed
Sprint 5	8	8	108	Completed
Sprint 6	8	0	108	Active
Sprint 7	10	0	100	Upcoming
Sprint 8	8	0	92	Upcoming
Sprint 9	8	0	84	Upcoming
Sprint 10	10	0	74	Upcoming
Sprint 11	8	0	66	Upcoming
Sprint 12	8	0	58	Upcoming
Sprint 13	6	0	52	Upcoming
Sprint 14	6	0	46	Upcoming
Sprint 15	5	0	41	Upcoming
Sprint 16	8	0	33	Upcoming
Sprint 17	8	0	25	Upcoming
Sprint 18	6	0	19	Upcoming
Sprint 19	5	0	14	Upcoming
Sprint 20	4	0	10	Upcoming

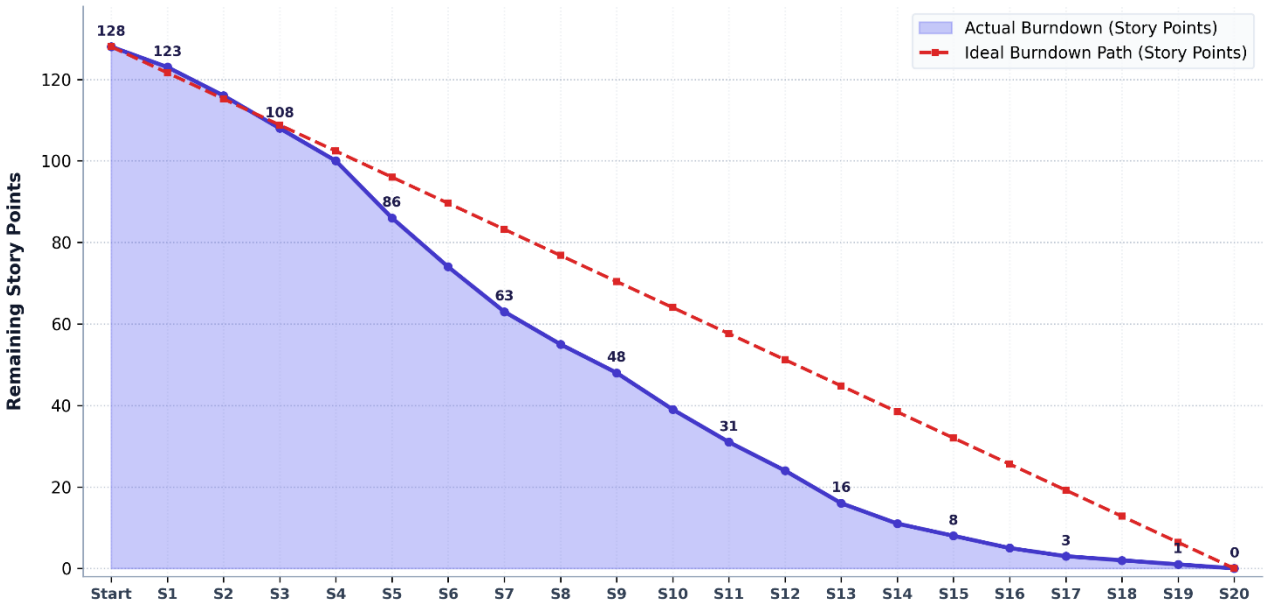


Figure 10*: FL-KubeOps 20-Sprint Burndown Path*

5.5 Backlog Refinement Plan

Frequency: Once per sprint (every 2 weeks), held mid-sprint (Wednesday of week-2)

Process:

- **Review K3s cluster behavior:** Check for operator crashes, resource exhaustion on commodity laptops, and PySyft compatibility issues.
- **Assess PySyft compatibility:** Verify SMPC primitives work with the Python version pinned in the Docker images; update if upstream breaking changes occur.
- **Advisor feedback integration:** Prof Dr. Saira Andleeb Gillani reviews sprint demos and sprint demos and suggests reprioritization; high-priority feedback creates new backlog items for the next sprint.
- **Definition of ready update:** Ensure all backlog items entering the next sprint have acceptance criteria, story point estimates, and assigned focal persons.

6. Risk Analysis Upcoming

Risk ID	Description	Impact	Mitigation Strategy
R-01	PySyft SMPC compatibility issues with latest Python/Kubernetes versions	High	Pin PySyft and Python versions in Docker images; maintain a compatibility matrix.
R-02	K3s resource constraints on team hardware (commodity laptops)	Medium	Limit client nodes to 4; use CPU-only PyTorch.
R-03	Limited availability of federated healthcare benchmark datasets in Pakistan	Medium	Generate high-fidelity synthetic patient data using Bayesian network models conditioned on NHANES feature distributions.
R-04	Complexity of kopf operator reconciliation loops	High	Start with a minimal reconciliation loop.
R-05	Regulatory ambiguity in Pakistan's health data policy	Medium	Design system to exceed current draft requirements.
R-06	Team skill gap in Kubernetes operator patterns	Medium	kopf tutorial series in Sprint 0; pair programming fallback to Helm charts
R-07	Mid-round node failure during demo/evaluation	High	Implement deterministic checkpointing every 2 rounds.
R-08	Kubeflow Pipelines SDK breaking changes	Medium	Pin SDK version in requirements.txt; subscribe to Kubeflow release notes.
R-09	Scope creep , adding too many aggregation algorithms instead of focusing on FedAvg + SMPC	High	Strict sprint gate: no new aggregation strategies until CC-3 (SMPC) and CC-6 (Benchmarking) are complete.
R-10	Third-party deployment failure during reproducibility testing	Medium	Write deployment guide incrementally (Sprint 17); test with a team member who did not write the guide (Sprint 18).

7. References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS), Fort Lauderdale, FL, 2017, pp. 1273–1282.
[2] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS), Dallas, TX, 2017, pp. 1175–1191.
[3] M. Abadi et al., "Deep learning with differential privacy," in Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS), Vienna, Austria, 2016, pp. 308–318.
[4] T. Li, A. K. Diao, V. Smith, Y. Papailiopoulos, and V. Smola, "Federated learning: Challenges, methods, and future directions," IEEE Signal Process. Mag., vol. 37, no. 3, pp. 50–60, May 2020.
[5] N. Rieke et al., "The future of digital health with federated learning," NPJ Digit. Med., vol. 3, no. 1, pp. 1–7, 2020.
[6] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv preprint arXiv:2007.14390, 2020.
[7] G. A. Reina et al., "OpenFL: An open-source framework for federated learning," arXiv preprint arXiv:2105.06413, 2021.
[8] T. Li, A. Hu, A. Babakniya, C. Ma, and M. Sanjabi, "FedProx: Tackling heterogeneity in federated learning," arXiv preprint arXiv:1812.06127, 2020.
[9] The Kubeflow Authors, Kubeflow: Machine Learning Toolkit for Kubernetes, v1.8, 2024. [Online]. Available: https://www.kubeflow.org/docs/
[10] Zalando SE, kopf: Kubernetes Operator Pythonic Framework, v1.37, 2024. [Online]. Available: https://kopf.readthedocs.io/
[11] The Linux Foundation, K3s: Lightweight Kubernetes, 2024. [Online]. Available: https://k3s.io/
[12] MLflow Authors, MLflow: An Open Source Platform for the Machine Learning Lifecycle, v2.x, 2024. [Online]. Available: https://mlflow.org/
[13] OpenMined, PySyft: A system for privacy-preserving AI, 2024. [Online]. Available: https://github.com/OpenMined/PySyft
[14] J. Dobies and J. Wood, Kubernetes Operators: Automating the Container Orchestration Platform, O'Reilly Media, 2021.
[15] L. Sweeney, "Simple demographics often identify people uniquely," Carnegie Mellon University, Data Privacy Working Paper 3, Pittsburgh, PA, 2000.
[16] Grand View Research, Privacy Enhancing Technologies Market Report, 2024-2030, 2024.
[17] Precedence Research, Privacy Preserving AI Market Size to Hit USD 39.93 Billion by 2035, 2026.
[18] Future Market Insights, Federated Learning Healthcare Market Analysis, 2026

Appendix A: Glossary

Term	Definition
Federated Learning (FL)	A distributed machine learning paradigm where models are trained across decentralized devices holding local data samples, without exchanging raw data.
Secure Multiparty Computation (SMPC)	A cryptographic protocol allowing multiple parties to jointly compute a function over their inputs while keeping those inputs private.
Kubernetes Operator	A method of packaging, deploying, and managing a Kubernetes application using custom resources and controller logic.
Custom Resource Definition (CRD)	A Kubernetes API extension that defines a new object type (e.g., FLCluster) stored in the etcd database.
Kopf	A Python framework for building Kubernetes operators using decorators and asynchronous event handling.
Kubeflow Pipeline	A platform for building and deploying portable, scalable machine learning workflows based on Docker containers and Kubernetes.
K3s	A lightweight, certified Kubernetes distribution designed for resource-constrained environments and edge computing.
MLflow	An open-source platform for managing the end-to-end machine learning lifecycle, including experiment tracking and model versioning.
PySyft	An open-source framework by OpenMined for privacy-preserving AI, providing SMPC, differential privacy, and federated learning primitives.
FLCluster	The custom resource defined by FL-KubeOps representing a federated learning deployment (clients, server, strategy, and security parameters).
Gradient Aggregation	The process of combining model updates (gradients) from multiple clients into a single global update.
Data Sovereignty	The principle that data is subject to the laws and governance structures of the nation or facility where it is physically located.
Differential Privacy	A mathematical framework for sharing information about a dataset by describing patterns of groups while withholding information about individuals.
Homomorphic Encryption	A form of encryption permitting computation on ciphertexts, generating encrypted results that decrypt to match plaintext computation results.
Re-identification Attack	A technique where anonymized datasets are linked with auxiliary data to re-identify specific individuals.
MLOps	A set of practices for collaboration and communication between data scientists and operations professionals to manage production ML lifecycles.
DAG (Directed Acyclic Graph)	A graph structure used in pipeline frameworks where nodes represent tasks and edges represent dependencies, with no cyclic dependencies.
Reconciliation Loop	The core control mechanism of a Kubernetes operator that continuously compares desired state (CRD spec) with observed state (cluster reality) and acts to converge them.

Appendix B: IV & V Report

(Independent verification & validation)
IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
…					

Table 1: List of non-trivial defects

University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

Faculty of Information Technology and Computer Science

Semester wise SDP Meeting Report

Group ID	**Student Roll Number**		**Student Name and Signatures**	**Advisor**

Sr.	Date	Status (P/A/L)				Agenda Items	Notes
		Student 1	Student 2	Student 3	Student 4		
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							

Date: Advisor's Signatures: _____